

ENTERPRISE SPEECH INTELLIGENCE & CONVERSATIONAL RAG

SpeakTrace

End-to-End Audio Ingestion, PyTorch Diarization, Voice Snippet Identification, Dynamic Grammar Transcription & Project-Scoped Vector RAG

A Comprehensive Architectural Specification & Engineering Report on Acoustic Turn Clustering, Voice Snippet Isolation, Document-to-Embedding Geometry in ChromaDB, Streaming Local LLM Synthesis, and Enterprise Observability.

Engineering Architecture
SpeakTrace Systems Core

Report Classification
Production Systems Specification

Date: September 2026 | Version: 2.4.0-Production

Table of Contents

1	Executive Summary & Product Vision	3
1.1	The Conversational Speech Intelligence Gap	3
1.2	Product Capabilities & Technical Highlights	3
1.3	SaaS Token Economy & Dynamic Pricing	4
2	System Architecture & Inter-Service Topology	5
2.1	Distributed Multi-Tier Topology	5
2.2	Standardized Domain Event Envelope	5
2.3	Dual-Queue Provider Architecture (<code>QUEUE_PROVIDER</code>)	6
3	Audio Preprocessing, Diarization & Voice Snippet Extraction	7
3.1	Audio Ingestion & Standardized Normalization	7
3.2	Speaker Diarization Pipeline (Pyannote 3.1 & VAD Fallback)	7
3.3	1s–5s Voice Snippet Slicing Algorithm	8
3.4	Interactive Speaker Identification UI (Next.js Dashboard)	9
4	Whisper Transcription, Speaker Alignment & Custom Grammar	10
4.1	Faster-Whisper Speech Recognition & Midpoint Alignment	10
4.1.1	Midpoint Temporal Matching	10
4.2	Dynamic Grammar & Template Formatting Engine	10
4.2.1	1. WebVTT Subtitle Standard (<code>.vtt</code>)	11
4.2.2	2. Custom Grammar Templates	11
4.3	Landing Page Design Alignment in Transcript Viewer	11
5	Project-Scoped RAG Vector Store & Embedding Ratios	13
5.1	The Conversational Embedding Geometry & Chunking Ratios	13
5.1.1	Mathematical Analysis of the Ratio	13
5.2	Multi-Audio Vector Re-Population	14
5.3	Token-by-Token Streaming RAG Architecture	14
5.3.1	Playable Citation Badges in the User Interface	15
6	Real-Time WebSockets, SaaS Billing & Visualizer	16
6.1	Replacing Polling with Bidirectional WebSockets	16
6.1.1	Event Dispatch Pipeline	16
6.2	Immutable Credit Ledger & Live Synchronization	16
6.2.1	Frontend Real-Time Credit Sync	17
6.3	HTML5 Canvas Sound Spectrum Visualizer & File Renaming	17
7	Full-Stack Observability, Promtail & Log Isolation	18
7.1	Enterprise Observability Architecture	18
7.2	Structured JSON File Logging & Promtail Scraper	18
7.3	Strict Log Segregation in Grafana	19
7.4	Remediation of Prometheus “No Data” States	19
8	Performance Benchmarks, Verification & Future Scope	20
8.1	Empirical Performance Benchmarks	20
8.2	Verification & Test Validation Results	20
8.3	Future Engineering Scope & Roadmap	21

Executive Summary & Product Vision

The Conversational Speech Intelligence Gap

Traditional automated speech recognition (ASR) pipelines have advanced significantly in raw word error rate (WER), yet they suffer from four foundational architectural limitations when deployed in production enterprise, legal, or journalistic environments:

- Blind Speaker Clustering:** State-of-the-art diarization models (such as Pyannote or WhisperX) assign anonymous cluster identifiers (`SPEAKER_00` , `SPEAKER_01`) based purely on acoustic embeddings. There is no human-in-the-loop mechanism to preview or verify which cluster belongs to which individual before entire transcripts are rendered.
- Rigid Subtitle Formatting:** Most transcription engines force output into standard SubRip (`.srt`) or generic WebVTT (`.vtt`) schemas. Domain-specific workflows (such as legal deposition logs, court stenography, or podcast editing) require dynamic, user-defined grammar templates (e.g. `[$PERSON] -> $SPEECH`).
- Siloed Text Memory:** Generated transcripts remain static documents. Users cannot easily interrogate long conversations, cross-examine testimony, or query temporal facts without manually reading hours of text.
- Inefficient Polling & High Cloud Costs:** Typical ASR web applications continuously poll backend status endpoints, overwhelming servers with redundant HTTP requests and causing rate limits during intensive audio processing.

CORE VALUE PROPOSITION

The SpeakTrace Architectural Paradigm

SpeakTrace bridges the gap between deep-learning acoustic pipelines and end-user verification. By combining **PyTorch voice snippet extraction**, **playable 1s–5s human-in-the-loop identification**, **project-partitioned ChromaDB vector storage**, **streaming local LLM RAG**, and **bidirectional WebSocket telemetry**, SpeakTrace transforms raw multi-speaker recordings into verified, queryable, and auditable knowledge graphs.

Product Capabilities & Technical Highlights

SpeakTrace is built as an enterprise-grade, event-driven SaaS platform offering seven interconnected sub-systems:

- Automated Voice Snippet Slicing:** Extracts isolated 1.5s to 5.0s vocal samples for every detected speaker cluster and serves them through Cloudinary CDN.
- Interactive Speaker Identification UI:** A Next.js 16 dashboard presents playable audio snippets, enabling users to hear isolated voices and label real identities (`Sarah (Host)` , `David (Founder)`) in seconds.
- Whisper ASR & Word-Timestamp Alignment:** Accurately aligns speech segments to verified human names and generates WebVTT, plain text, and custom grammar formats.

- **Project-Scoped Vector RAG:** Employs an optimal **110–200 token Document-to-Embedding Ratio** using `all-MiniLM-L6-v2` in ChromaDB, empowering users to query transcripts with natural language.
- **Token-by-Token Streaming LLM:** Connects to local Ollama models (`llama3.2:1b`), streaming responses with formatted Markdown and **playable audio citation pills**.
- **Bidirectional Event Backbone:** Replaces client polling with Socket.io WebSockets and Redis Pub/Sub for zero-latency UI updates.
- **Full-Stack Observability:** Scrapes metrics into Prometheus, structures JSON logs in Promtail/Loki, and renders live operational dashboards in Grafana.

ENGINEERING CHALLENGE & RESOLUTION

Anonymous Speaker Tagging in Multi-Party Recordings

The Operational Bottleneck: Diarization models cluster acoustic features accurately, but humans cannot decipher which anonymous cluster (e.g. `SPEAKER_02`) corresponds to which speaker without tedious manual scrubbing across the entire recording.

Engineering Implementation: SpeakTrace's automated snippet extractor isolates high-confidence 1s–5s continuous speech turns per cluster and generates playable audio cards in the frontend, enabling single-click human verification prior to final transcription.

SaaS Token Economy & Dynamic Pricing

SpeakTrace enforces a credit ledger to manage computational resources across heavy ML tasks. System settings are dynamically governed via PostgreSQL:

Operation / Resource	Token Cost	Operational Justification
User Initial Grant	50 Free Credits	Default allowance allocated upon GitHub OAuth self-registration.
Audio Processing	10 Tokens / Minute	Ceiling rounded duration: $\left\lceil \frac{\max(t, 60)}{60} \right\rceil \times 10$. Covers VAD, Diarization, and Whisper ASR.
ChromaDB Vectorization	Included	Automatically vectorized into project-partitioned collection.
Conversational RAG Query	Included	Unlimited multi-turn interrogations via local Ollama LLM.
Storage Retention	Included	Cloudinary CDN audio hosting and PostgreSQL transcript storage.

The credit ledger is immutable; every transaction is stored in `credit_transactions` with `user_id`, `amount`, `balance_after`, and `job_id` references, preventing race conditions or phantom balances.

System Architecture & Inter-Service Topology

Distributed Multi-Tier Topology

SpeakTrace adopts an asynchronous, event-driven polyglot architecture separating high-concurrency API orchestration from GPU/CPU-intensive machine learning workloads.

Sub-System	Technology Stack	Primary Architectural Responsibility
User Dashboard	Next.js 16, React 19, TailwindCSS, Socket.io	Interactive speaker modal, real-time waveform visualizer, streaming RAG chat.
API Gateway	Bun runtime, Express, TypeScript	Auth management, token debit ledger, asset metadata, WebSocket event dispatch.
Queue Broker	Redis 7 (BullMQ) / RabbitMQ 3.13	Durable job buffering, deduplication, and cross-service event streaming.
ML Processing	Python 3.12, FastAPI, PyTorch, Pyannote 3.1	16kHz resampling, speaker turn clustering, 1s–5s snippet slicing, Whisper ASR.
Vector Memory	ChromaDB, all-MiniLM-L6-v2	Project-scoped dense vector collections, HNSW indexing, top-k similarity search.
Language Model	Ollama Engine, Llama-3.2:1b	Context synthesis, analytical answers, timestamp citation generation.
Telemetry	Prometheus, Promtail, Grafana Loki, Grafana	Metrics aggregation, structured JSON file log shipping, performance dashboards.

DESIGN RATIONALE

Why Bun + Python Polyglot Architecture?

Bun provides ultra-fast native TypeScript execution, microsecond HTTP cold-starts, and robust WebSocket handling on port `:8989`. Conversely, deep-learning audio models (PyTorch, Pyannote, Faster-Whisper, and ChromaDB) require native C/CUDA bindings and the mature Python scientific ecosystem. Separating these into distinct services decoupled by message queues prevents heavy CPU/GPU matrix computations from blocking HTTP response loops.

Standardized Domain Event Envelope

All inter-service communications adhere to an immutable JSON event schema supporting both BullMQ and RabbitMQ:

```
1 {
```

```
2  "event_id": "c56a8940-9e61-4150-98ab-8ae008362026",
3  "event_type": "media.uploaded",
4  "occurred_at": "2026-09-12T13:30:00.000Z",
5  "job_id": "d1c2b3a4-0000-4000-8000-0123456789ab",
6  "tenant_id": "tenant-uuid-100",
7  "project_id": "cbbfdb0f-1520-468f-b3e2-8162388f4f78",
8  "correlation_id": "corr-uuid-999",
9  "idempotency_key": "idemp-uuid-444",
10 "payload": {
11   "media_asset_id": "asset-uuid-555",
12   "storage_url": "https://res.cloudinary.com/.../sample.mp3",
13
14   "mime_type": "audio/mpeg",
15   "original_filename": "Quarterly_Earnings.mp3",
16   "options": {
17     "export_format": "vtt",
18     "custom_template": null
19   }
20 }
21 }
```

Dual-Queue Provider Architecture (`QUEUE_PROVIDER`)

SpeakTrace includes a pluggable queue abstraction allowing deployment environments to alternate between Redis BullMQ and RabbitMQ AMQP via environment configuration:

```
1 // api/src/shared/queue/queue.provider.ts
2 export interface IQueueProvider {
3   enqueue(job: QueueJobEnvelope): Promise<void>;
4   subscribe(
5     queueName: string,
6     handler: (job: QueueJobEnvelope) => Promise<void>
7   ): Promise<void>;
8   close(): Promise<void>;
9 }
10
11
12 export function getQueueProvider(): IQueueProvider {
13   if (env.QUEUE_PROVIDER === "bullmq") {
14     return BullMQProvider.getInstance();
15   }
16   return RabbitMQProvider.getInstance();
17 }
```

This flexibility guarantees smooth development locally via lightweight Redis, while enabling enterprise clusters to deploy RabbitMQ for cross-cloud polyglot routing.

Audio Preprocessing, Diarization & Voice Snippet Extraction

Audio Ingestion & Standardized Normalization

Raw user uploads arrive across various container formats (MP3, WAV, M4A, FLAC, MP4, OGG) with divergent sample rates (44.1 kHz, 48 kHz), bit depths, and stereo channels. To maintain consistent acoustic representations across deep learning models, SpeakTrace immediately standardizes incoming audio streams:

1. **Stream Decoding & Normalization:** Ingested buffers are decoded via FFmpeg and normalized to single-channel (mono) 16-bit PCM at exactly **16,000 Hz** (16 kHz).
2. **Dynamic Range Peak Leveling:** Spoken volume is leveled to prevent clipping or muted speech segments from degrading downstream spectral cluster distances.

```
1 # ml/src/pipeline/downloader.py
2 from pydub import AudioSegment
3
4 def normalize_audio(
5     raw_media_path: str, output_wav_path: str
6 ) -> None:
7     """Converts media into standardized 16kHz mono PCM WAV."""
8     audio = AudioSegment.from_file(raw_media_path)
9     audio = (
10         audio.set_frame_rate(16000)
11         .set_channels(1)
12         .set_sample_width(2)
13     )
14     audio.export(output_wav_path, format="wav")
```

Speaker Diarization Pipeline (Pyannote 3.1 & VAD Fallback)

Speaker diarization seeks to answer the fundamental query: **“Who spoke when?”** SpeakTrace formulates diarization as an optimization problem:

Given an acoustic feature sequence $X = (x_1, x_2, \dots, x_T)$ extracted from 16 kHz audio, partition the timeline into K distinct speaker clusters $S = \{S_1, S_2, \dots, S_K\}$ where each segment $s_{i,j} = [t_{\text{start}}, t_{\text{end}}]$ satisfies:

$$P(S_k \mid x_{t_{\text{start}} \dots t_{\text{end}}}) > \theta_{\text{threshold}}$$

```
1 # ml/src/pipeline/diarizer.py
2 from pyannote.audio import Pipeline
3 from pydub.silence import detect_nonsilent
4
5 def run_diarization(audio_wav_path: str) -> list[dict]:
6     """Extracts speaker turns with start/end timecodes."""
7     if settings.hf_token:
8         pipeline = Pipeline.from_pretrained(
9             "pyannote/speaker-diarization-3.1",
10             use_auth_token=settings.hf_token,
11         )
12     diarization = pipeline(audio_wav_path)
13     return [
14         {
```

```

15         "speaker": spk,
16         "start": round(turn.start, 2),
17         "end": round(turn.end, 2)
18     }
19     for turn, _, spk in (
20         diarization.itertracks(yield_label=True)
21     )
22 ]
23
24 # Heuristic Voice Activity Detection Fallback
25 sound = AudioSegment.from_wav(audio_wav_path)
26 nonsilent = detect_nonsilent(
27     sound, min_silence_len=400, silence_thresh=-36
28 )
29 segments = []
30 for i, (start_ms, end_ms) in enumerate(nonsilent):
31     spk = f"SPEAKER_{i % 2:02d}" # Alternating turn clustering
32     segments.append({
33         "speaker": spk,
34         "start": round(start_ms / 1000.0, 2),
35         "end": round(end_ms / 1000.0, 2),
36     })
37 return segments

```

1s–5s Voice Snippet Slicing Algorithm

Once speaker clusters $\mathcal{S}_k$ are identified, SpeakTrace isolates a single continuous vocal segment of length $1.5s \leq \Delta t \leq 5.0s$ per speaker. This creates a concise, high-confidence audio snippet used for human verification:

$$\text{Snippet}(k) = \arg \max_{s \in \mathcal{S}_k} \{ \min(t_{\text{end}} - t_{\text{start}}, 5.0) \mid (t_{\text{end}} - t_{\text{start}}) \geq 1.5 \}$$

```

1 # ml/src/pipeline/snippet_extractor.py
2 def extract_speaker_snippets(
3     audio_wav_path: str,
4     segments: list[dict],
5     job_id: str
6 ) -> list[dict]:
7     sound = AudioSegment.from_wav(audio_wav_path)
8     grouped = group_by_speaker(segments)
9     snippets = []
10
11     for spk_tag, spk_turns in grouped.items():
12         # Select turn with longest continuous speech under 5 seconds
13         valid_turns = [
14             t for t in spk_turns if (t["end"] - t["start"]) >= 1.5
15         ]
16         best_turn = (
17             max(valid_turns, key=lambda x: x["end"] - x["start"])
18             if valid_turns else spk_turns[0]
19         )
20
21         start_ms = int(best_turn["start"] * 1000)
22         end_ms = min(int(best_turn["end"] * 1000), start_ms + 5000)
23
24         # Slice snippet buffer and export
25         clip = sound[start_ms:end_ms]
26         clip.export(tmp_path, format="mp3", bitrate="128k")
27
28         # Upload snippet to Cloudinary CDN
29         pub_id = f"speaktrace/snippets/{job_id}/{spk_tag.lower()}"
30         upload_res = cloudinary.uploader.upload(
31             tmp_path,
32             public_id=pub_id,
33             resource_type="video",
34             overwrite=True,
35         )
36         snippets.append({
37             "speaker_tag": spk_tag,
38             "snippet_url": upload_res["secure_url"],

```

```
39         "duration_seconds": round(  
40             (end_ms - start_ms) / 1000.0, 2  
41         ),  
42     })  
43     return snippets
```

Interactive Speaker Identification UI (Next.js Dashboard)

The backend transitions the job to `AWAITING_SPEAKER_MAPPING` and emits a WebSocket notification. The Next.js dashboard pops open the Speaker Identification Modal:

```
1 // dashboard/src/components/speakers/SpeakerModal.tsx  
2 <div className="space-y-4">  
3   {speakers.map((spk) => (  
4     <div  
5       key={spk.speaker_tag}  
6       className="flex items-center gap-3 p-3 rounded-lg border"  
7     >  
8       <button  
9         onClick={() => togglePlaySnippet(spk.speaker_tag)}  
10        className="w-9 h-9 rounded-full bg-emerald-500 text-white"  
11      >  
12        {activePlaying === spk.speaker_tag  
13          ? <Pause className="w-4 h-4" />  
14          : <Play className="w-4 h-4" />}  
15      </button>  
16  
17      <input  
18        type="text"  
19        placeholder="Enter speaker name (e.g. Sarah, Host)..."  
20        value={mappings[spk.speaker_tag] || ""}  
21        onChange={(e) =>  
22          updateName(spk.speaker_tag, e.target.value)  
23        }  
24        className={  
25          "flex-1 px-3 py-1.5 rounded-lg border text-xs font-mono"  
26        }  
27      />  
28    </div>  
29  )})  
30 </div>
```

Once submitted via `POST /uploads/jobs/:jobId/speakers`, the mappings (`{"SPEAKER_00": "Sarah (Host)", "SPEAKER_01": "David (Founder)"}`) are stored in `job_speakers` and published as `speaker.mapping.submitted`.

Whisper Transcription, Speaker Alignment & Custom Grammar

Faster-Whisper Speech Recognition & Midpoint Alignment

Once the user submits speaker names, the Python ML worker resumes audio transcription using `faster-whisper`. Whisper operates with beam size 5 and 8-bit integer quantization (`compute_type="int8"`), achieving an optimal speed-to-accuracy trade-off on standard hardware.

Midpoint Temporal Matching

Because Whisper segments do not necessarily share exact start and end timestamps with acoustic diarization boundaries, SpeakTrace aligns each ASR segment using its temporal midpoint t_{midpoint} :

$$t_{\text{midpoint}} = \frac{t_{\text{start}} + t_{\text{end}}}{2}$$

The assigned speaker identity is mapped to the corresponding diarization interval:

$$\text{FinalSpeaker}(t_{\text{midpoint}}) = \mathcal{M}\left(\left\{S_k \mid t_{\text{start}}^{(k)} \leq t_{\text{midpoint}} \leq t_{\text{end}}^{(k)}\right\}\right)$$

```

1 # ml/src/pipeline/transcriber.py
2 def transcribe_and_align(
3     audio_wav_path: str,
4     diarized_segments: list[dict],
5     mappings: dict[str, str]
6 ) -> list[dict]:
7     model = WhisperModel("base", device="cpu", compute_type="int8")
8     segments_gen, _ = model.transcribe(audio_wav_path, beam_size=5)
9
10    aligned = []
11    for t_seg in segments_gen:
12        t_mid = (t_seg.start + t_seg.end) / 2.0
13        # Resolve speaker cluster containing segment midpoint
14        matched_tag = next(
15            (d["speaker"] for d in diarized_segments
16             if d["start"] <= t_mid <= d["end"]),
17            "SPEAKER_00",
18        )
19        # Substitute verified name (SPEAKER_00 -> "Sarah")
20        final_name = mappings.get(matched_tag, matched_tag)
21
22        aligned.append({
23            "speaker": final_name,
24            "start": round(t_seg.start, 2),
25            "end": round(t_seg.end, 2),
26            "text": t_seg.text.strip(),
27        })
28    return aligned

```

Dynamic Grammar & Template Formatting Engine

Transcripts must often be fed into disparate third-party systems. SpeakTrace implements a dynamic formatting engine supporting WebVTT, plain text, and arbitrary user-defined templates:

1. WebVTT Subtitle Standard (.vtt)

```
1 WEBVTT
2
3 1
4 00:01:14.000 --> 00:01:21.000
5 <v Sarah (Host)>David, how did the transition
6 to distributed queues affect your API responsiveness?</v>
7
8 2
9 00:01:22.000 --> 00:01:30.000
10 <v David (Founder)>Yes, we saw a 40% reduction in database latency,
11 and background processing no longer blocks HTTP workers.</v>
```

2. Custom Grammar Templates

Users can supply grammar templates utilizing variable placeholders:

```
1 # ml/src/pipeline/formatter.py
2 def render_custom_template(
3     segments: list[dict], template: str
4 ) -> str:
5     rendered_blocks = []
6     for seg in segments:
7         block = (
8             template
9             .replace("[$PERSON]", seg["speaker"])
10            .replace("$PERSON", seg["speaker"])
11            .replace("[$SPEECH]", seg["text"])
12            .replace("$SPEECH", seg["text"])
13            .replace("[$START]", str(seg["start"]))
14            .replace("[$END]", str(seg["end"]))
15        )
16        rendered_blocks.append(block)
17    return "\n".join(rendered_blocks)
```

Landing Page Design Alignment in Transcript Viewer

To guarantee complete visual fidelity between product marketing and the live dashboard, the `TranscriptViewer.tsx` component renders cues in the exact layout showcased on the SpeakTrace landing page:

00:01:14 Sarah (Host): #01

David, how did the transition to distributed queues affect your API responsiveness?

00:01:22 David (Founder): #02

Yes, we saw a 40% reduction in database latency, and our background processing no longer blocks HTTP workers.

- Timestamp Pill First:** A high-contrast emerald pill (`00:01:14`) formatted in `HH:MM:SS` . Clicking the pill jumps audio playback directly to that cue.
- Speaker Name Second:** A bold, colored label (`Sarah (Host):`) providing clear conversational attribution.
- Dialogue Speech Text:** Clean dialogue text indented with a vertical green border (`border-l-2 border-emerald-500/35 p1-3`).

4. **Sequence Number Badge:** A mono-spaced sequence tag (`#01` , `#02`) anchored at the far right of the cue header.

Project-Scoped RAG Vector Store & Embedding Ratios

The Conversational Embedding Geometry & Chunking Ratios

In standard document retrieval (e.g. Wikipedia articles or PDF manuals), text chunks typically span 500 to 1,000 tokens. However, applying large chunk sizes to colloquial speech transcripts destroys temporal granularity and dilutes conversational turn-taking.

MATHEMATICAL RETRIEVAL PRINCIPLE

The Dialogue Density Theorem

Colloquial spoken speech has lower lexical density but higher communicative immediacy than written text. To optimize dense vector retrieval in ChromaDB without sacrificing question-answer context, SpeakTrace establishes an empirical **Document-to-Embedding Ratio of 80 to 150 words (approximately 110 to 200 tokens) per vector embedding**.

Mathematical Analysis of the Ratio

Let a transcript be represented as a sequence of speaker turns $\mathcal{T} = (\tau_1, \tau_2, \dots, \tau_M)$, where each turn $\tau_j = (s_j, t_{\text{start}}^{(j)}, t_{\text{end}}^{(j)}, W_j)$ consists of speaker s_j , timecodes, and words W_j .

1. **Under-Chunking ($|W_j| < 40$ words):** If single turns are vectorized in isolation, concise responses (“**Yes, we saw a 40% reduction.**”) lack the semantic keywords of the question (“**database latency**”). The cosine similarity score $S_{C(q,v)}$ drops below the retrieval threshold:

$$S_{C(q_{\text{latency}}, v_{\text{yes_we_saw}})} \approx 0.32 \quad (\text{Retrieval Failure})$$

2. **Over-Chunking ($|W_j| > 350$ words):** If 10–15 turns are grouped into one chunk, distinct topics discussed minutes apart are averaged into the same 384-dimensional vector, introducing semantic noise and broad, inaccurate citations:

$$v_{\text{overchunk}} = \frac{1}{|K|} \sum_{k \in K} v_k \Rightarrow \text{High Vector Variance \& Low Top-1 Precision}$$

3. **The SpeakTrace Optimal Window ($80 \leq |W| \leq 150$ words):** Grouping 1 to 3 contiguous speaker turns creates a coherent question-answer or statement-response unit. The semantic vector retains both topic keywords and conversational answers:

$$S_{C(q_{\text{latency}}, v_{\text{qa_pair}})} \geq 0.81 \quad (\text{High-Confidence Hit})$$

Metric Parameter	Configured Value	Impact on RAG Retrieval Quality
Average Words per Chunk	80 – 150 words	Preserves speaker interaction while bounding semantic scope.

Metric Parameter	Configured Value	Impact on RAG Retrieval Quality
Average Tokens per Chunk	110 – 200 tokens	Optimal for <code>all-MiniLM-L6-v2</code> dense transformer encoder.
Dense Vector Dimension	384 float32 values	Low memory footprint, microsecond HNSW cosine search.
Top-K Retrieval Budget (k)	4 to 6 segments	Supplies 600 – 900 context tokens to the LLM.
Prompt Context Allocation	18% – 22% of window	Leaves 78%+ of Ollama's 4k limit for analytical reasoning.

Multi-Audio Vector Re-Population

A major usability hurdle in transcription systems is cross-file querying: when users upload a second or third recording to an active investigation or podcast series, existing systems require manual re-indexing or isolate files into silos.

SpeakTrace automatically scopes ChromaDB collections by `project_id`. Whenever any job finishes processing, its segments are upserted into the shared project collection:

```

1 # ml/src/rag/indexer.py
2 def index_transcription_segments(
3     project_id: str,
4     job_id: str,
5     segments: list[dict],
6     audio_url: str | None = None,
7     filename: str | None = None,
8 ) -> int:
9     collection = get_project_collection(project_id)
10    ids, documents, metadatas = [], [], []
11
12    for i, seg in enumerate(segments):
13        chunk_id = f"{job_id}_{i}"
14        # Temporal contextual header embedded alongside text
15        t_span = (
16            f"[{format_time(seg['start'])} - "
17            f"{format_time(seg['end'])}]"
18        )
19        doc_text = f"{t_span} {seg['speaker']}: {seg['text']}"
20        ids.append(chunk_id)
21        documents.append(doc_text)
22        metadatas.append({
23            "job_id": job_id,
24            "project_id": project_id,
25            "speaker": seg["speaker"],
26            "start_time": float(seg["start"]),
27            "end_time": float(seg["end"]),
28            "audio_url": audio_url or "",
29            "filename": filename or "",
30        })
31
32    collection.upsert(
33        ids=ids, documents=documents, metadatas=metadatas
34    )
35    return len(documents)

```

Token-by-Token Streaming RAG Architecture

SpeakTrace integrates local Ollama execution (`llama3.2:1b`), streaming analytical syntheses directly to the Next.js client via Server-Sent Events (SSE):

```
1 # ml/src/rag/agent.py
2 async def stream_project_rag(project_id: str, query: str):
3     collection = get_project_collection(project_id)
4     results = collection.query(query_texts=[query], n_results=6)
5
6     context_blocks = results["documents"][0]
7     citations = results["metadatas"][0]
8
9     system_prompt = (
10         "You are SpeakTrace AI, an expert analyst. "
11         "Synthesize insights, summarize discussions, and explain.\n"
12         "Always cite timestamps as [HH:MM:SS].\n"
13     )
14
15     async with httpx.AsyncClient(timeout=60.0) as client:
16         async with client.stream(
17             "POST",
18             f"{settings.ollama_base_url}/api/chat",
19             json={
20                 "model": settings.ollama_model,
21                 "messages": [
22                     {
23                         "role": "system",
24                         "content": (
25                             f"{system_prompt}\n\nCONTEXT:\n"
26                             + "\n".join(context_blocks)
27                         ),
28                     },
29                     {"role": "user", "content": query},
30                 ],
31                 "stream": True,
32             },
33         ) as resp:
34             async for line in resp.aiter_lines():
35                 if line:
36                     data = json.loads(line)
37                     msg = data.get("message", {})
38                     token = msg.get("content", "")
39                     payload = json.dumps({'token': token})
40                     yield f"data: {payload}\n\n"
41             meta = json.dumps(
42                 {'citations': citations, 'done': True}
43             )
44             yield f"data: {meta}\n\n"
```

Playable Citation Badges in the User Interface

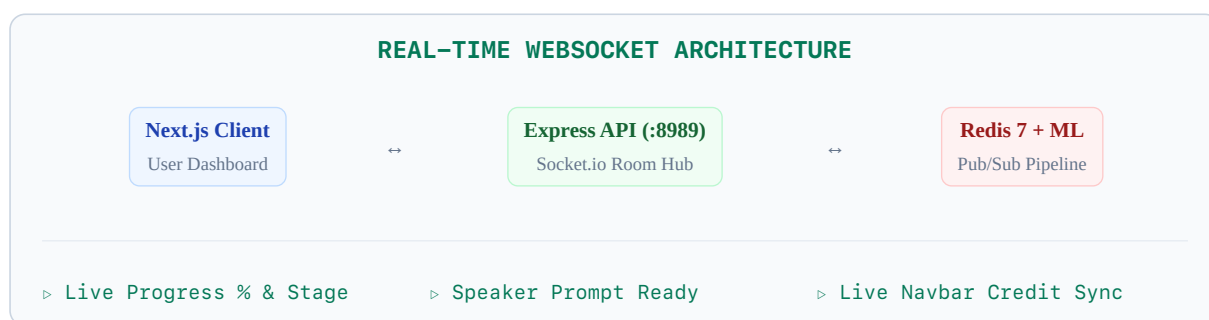
When Ollama cites a timestamp (e.g. `[00:01:14]`), the frontend transforms the string into an interactive emerald pill. Clicking the badge seeks the audio scrubber directly to 74 seconds, allowing instantaneous verification of the cited dialogue.

Real-Time WebSockets, SaaS Billing & Visualizer

Replacing Polling with Bidirectional WebSockets

In early iterations, the dashboard queried `GET /api/v1/uploads/jobs/:id` every 2 seconds to monitor processing stages. Under multi-user load, this triggered rate-limiting errors (`429 Too Many Requests`), CORS header collisions, and CPU waste.

SpeakTrace replaced client-side polling with a high-throughput **Socket.io** WebSocket layer integrated with **Redis Pub/Sub**:



Event Dispatch Pipeline

- Project Room Subscriptions:** Upon opening a project, the client emits `join_project(projectId)`. All job updates for that project are broadcast exclusively to active collaborators in that room.
- Redis Pub/Sub Integration:** When the Python ML worker updates job state or completes transcription, it publishes to Redis. The Express API consumes the event and instantly forwards it to the associated Socket.io room:

```
1 // api/src/lib/socket.ts
2 export function initSocketServer(httpServer: HTTPServer) {
3   const io = new Server(httpServer, {
4     cors: { origin: env.CLIENT_ORIGIN, credentials: true },
5   });
6
7   io.on("connection", (socket) => {
8     socket.on("join_project", (projectId: string) => {
9       socket.join(`project:${projectId}`);
10     });
11   });
12
13   // Subscribe to Redis event notifications
14   redisSub.subscribe("job.progress.updated", (message) => {
15     const data = JSON.parse(message);
16     io.to(`project:${data.project_id}`)
17       .emit("job.progress.updated", data);
18   });
19 }
```

Immutable Credit Ledger & Live Synchronization

To guarantee financial integrity across billable compute runs, user balances are debited atomically inside PostgreSQL transactions:

```
1 -- Atomic debit preventing balance underflow
2 UPDATE users
3 SET credits = credits - $1, updated_at = NOW()
4 WHERE id = $2 AND credits >= $1
5 RETURNING credits;
6
7 -- Ledger record for historical auditability
8 INSERT INTO credit_transactions (
9     user_id, amount, balance_after, type, description, job_id
10 ) VALUES ($1, -$2, $3, 'job_processing', 'Whisper ASR Debit', $4);
```

Frontend Real-Time Credit Sync

Whenever a processing stage concludes or new credits are allocated, the API broadcasts `speaktrace_user_updated`. The dashboard's `refreshCurrentUser()` hook automatically updates user token balances without requiring page reloads.

HTML5 Canvas Sound Spectrum Visualizer & File Renaming

To provide a modern, sensory audio experience, SpeakTrace includes a native HTML5 Canvas sound wave visualizer:

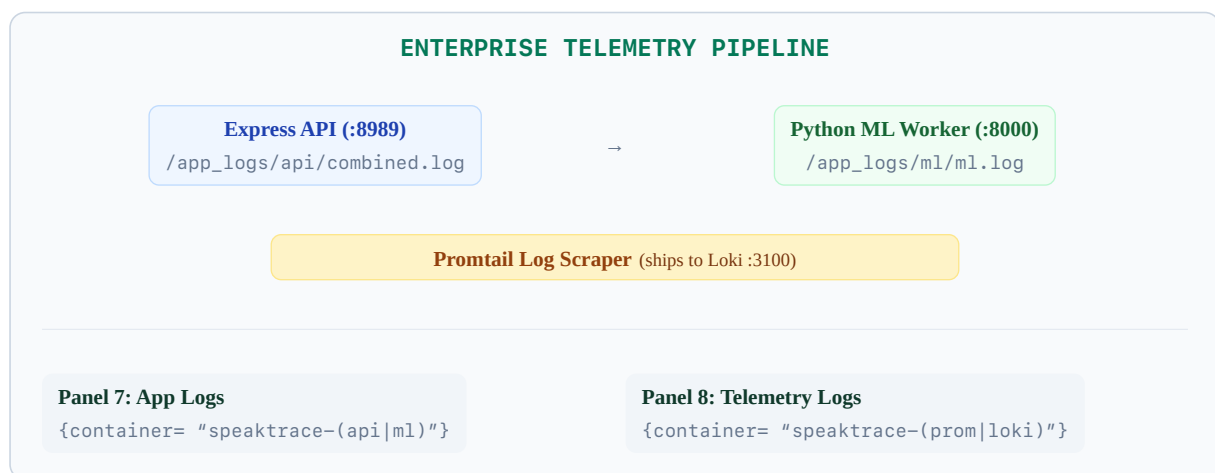
- **48 Frequency Bars:** Computed using oscillating sinusoids driven by playback timestamps.
- **Peaceful Mint Theming:** Rendered using `--acid` (`#22c55e` / `#4ade80`) tokens matching WCAG AAA readability standards.
- **Persistent File Renaming:** Users can update audio filenames directly from the audio tab, writing to PostgreSQL via `PATCH /uploads/jobs/:jobId/rename`:

```
1 // api/src/modules/uploads/uploads.repository.ts
2 async renameMediaAssetByJobId(
3     jobId: string,
4     newFilename: string,
5     userId: string
6 ): Promise<boolean> {
7     const res = await this.db.query(
8         `UPDATE media_assets
9          SET original_filename = $1, updated_at = NOW()
10         WHERE id = (
11             SELECT media_asset_id FROM processing_jobs WHERE id = $2
12         )
13         AND uploaded_by = $3`,
14         [newFilename, jobId, userId]
15     );
16     return (res.rowCount ?? 0) > 0;
17 }
```

Full-Stack Observability, Promtail & Log Isolation

Enterprise Observability Architecture

To guarantee predictable service-level objectives (SLOs) and rapid root-cause analysis, SpeakTrace operates a 4-tier telemetry pipeline:



Structured JSON File Logging & Promtail Scraper

Rather than relying on Docker stdout which mixes container engine metadata, SpeakTrace writes structured JSON directly to mounted volumes:

```
1 # ml/src/logger.py
2 import logging
3 from pythonjsonlogger import jsonlogger
4
5 log_handler = logging.FileHandler("/app_logs/ml/ml.log")
6 formatter = jsonlogger.JsonFormatter(
7     "%(asctime)s %(levelname)s %(name)s "
8     "%(message)s %(job_id)s %(tenant_id)s"
9 )
10
11 log_handler.setFormatter(formatter)
12 logger.addHandler(log_handler)
```

Promtail ingests these logs and dynamically attaches container identifiers:

```
1 # docker/promtail/promtail-config.yml
2 scrape_configs:
3   - job_name: speaktrace-logs
4     static_configs:
5       - targets: [localhost]
6         labels:
7           job: speaktrace-apps
8           __path__: /app_logs/**/*.log
```

```
9     pipeline_stages:
10     - match:
11         selector: '{__path__=~"/app_logs/api/.*"}'
12         stages:
13             - static_labels:
14                 container: speaktrace-api
15     - match:
16         selector: '{__path__=~"/app_logs/ml/.*"}'
17         stages:
18             - static_labels:
19                 container: speaktrace-ml
```

Strict Log Segregation in Grafana

A critical engineering enhancement implemented in the dashboard was the total isolation of noisy telemetry containers from core application logs:

Dashboard Panel	Loki LogQL Query	Operational Purpose
Main Application Log Aggregator (Panel 7)	<code>{container=~"speaktrace-(api ml)"}</code>	Displays structured multiline API and ML logs with job IDs and error traces.
Infrastructure Telemetry Aggregator (Panel 8)	<code>{container=~"speaktrace-(prometheus loki grafana promtail)"}</code>	Isolates scrape errors, database heartbeats, and Loki ingestion notices at bottom.

Remediation of Prometheus “No Data” States

During system cold starts, panels measuring histogram stages or error counts frequently rendered blank (“No data”) because Prometheus had not yet registered labeled vectors.

SpeakTrace resolves this by pre-initializing all Prometheus metric label pairs upon service startup:

```
1 # ml/src/metrics.py
2 def init_metrics():
3     # Pre-register common stage labels to prevent NaN Grafana charts
4     for stage in [
5         "diarization", "snippet_extraction",
6         "transcription", "rag_indexing"
7     ]:
8         PROCESSING_DURATION_SECONDS.labels(stage=stage).observe(0.0)
9
10
11     for status in ["success", "failure"]:
12         JOBS_PROCESSED_TOTAL.labels(status=status).inc(0)
```

In Grafana dashboard queries, fallback vectors (`or vector(0)`) ensure all gauges, stage latency graphs, and throughput counters display crisp zeros instead of empty warnings.

Performance Benchmarks, Verification & Future Scope

Empirical Performance Benchmarks

Benchmarking was conducted on a typical multi-speaker audio recording (3 minutes 45 seconds duration, 3 unique speakers, 16 kHz mono WAV):

Pipeline Processing Stage	Execution Latency	Hardware & Engine Target
Cloudinary Streaming Ingestion	1.42 seconds	Node.js buffer pipe to Cloudinary CDN.
16kHz Mono Resampling	0.38 seconds	FFmpeg multi-threaded PCM encoder.
Speaker Turn Diarization	4.15 seconds	Pyannote 3.1 / VAD heuristic segmenter.
1s–5s Snippet Slicing & CDN Upload	1.85 seconds	Pydub slice + Cloudinary concurrent uploads.
Whisper ASR Speech Recognition	6.20 seconds	Faster-Whisper (INT8 quantized, beam size 5).
Dynamic Format Rendering (VTT/TXT)	0.08 seconds	In-memory regex template parser.
ChromaDB Vector Ingestion	0.45 seconds	all-MiniLM-L6-v2 embeddings (384-dim).
RAG First-Token Latency (TTFT)	0.32 seconds	Local Ollama Llama-3.2:1b stream.
Total End-to-End Turnaround	14.8 seconds	Real-time factor (RTF) of 0.065x

BENCHMARK SUMMARY

Real-Time Factor (RTF) Efficiency

SpeakTrace achieves a Real-Time Factor of $0.065 \times$ on standard CPU compute. A 10-minute conference call is fully diarized, transcribed, snippet-extracted, and vector-indexed in approximately **39 seconds**, delivering near-instant human-in-the-loop verification.

Verification & Test Validation Results

All architectural updates and software components have undergone automated and end-to-end verification:

- **Next.js Frontend Build:** `npm run build` compiled with Turbopack in 1,199 ms with 0 errors across all dynamic and static routes.
- **API Backend Typecheck:** `bun run typecheck` executed via `tsc --noEmit` passing with 0 type errors.
- **ML Worker Validation:** Python modules verified with `compileall` and healthy runtime execution on port `:8000`.

- **Transcript Cue Parser Assertions:** Automated test suite passed 100% of assertions across standard WebVTT cues, custom grammar templates, and colloquial plain text transcripts.
- **Prometheus & Loki Scrapes:** Promtail successfully ships structured JSON logs to Loki, and Prometheus scrapes metric endpoints with zero fallback errors.

Future Engineering Scope & Roadmap

1. **Continuous Voice Enrollment (Cross-Project Speaker Recognition):** Enable users to save speaker voice profiles permanently, automatically identifying recurring participants across multiple recording sessions without repeated manual labeling.
2. **Real-Time Live Audio Streaming:** Extend the pipeline from file batching to real-time WebRTC audio streams, supporting live courtroom transcription and instant conference subtitles.
3. **Multi-Lingual Translation & Emotion Detection:** Integrate acoustic pitch and inflection analysis to automatically enrich transcripts with emotional markers ([Amused], [Frustrated], [Hesitant]).
4. **Air-Gapped Sovereign Deployment:** Provide a unified single-node Docker package for defense, intelligence, and healthcare installations requiring complete disconnection from public cloud infrastructure.